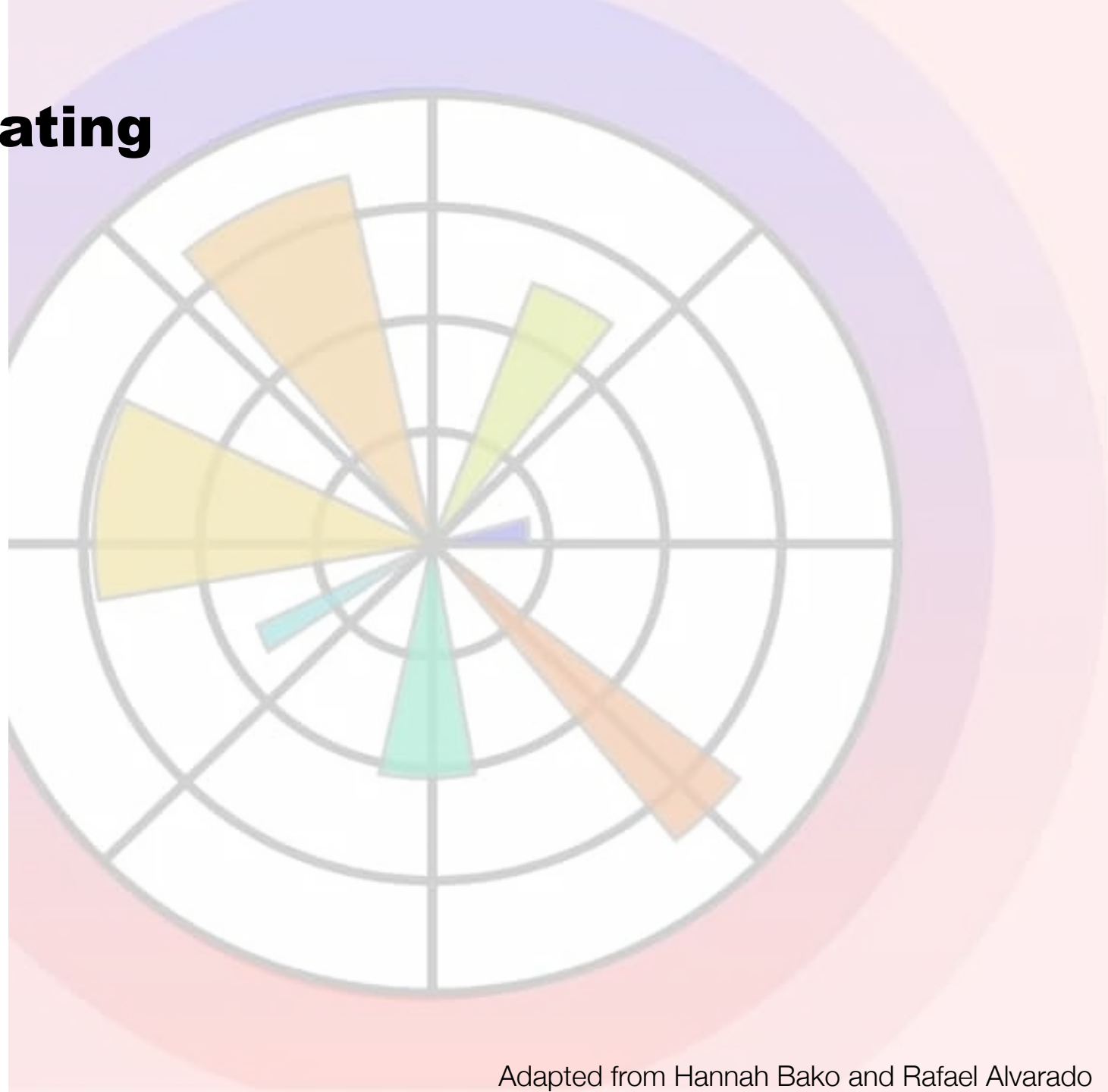


UVA DS2003 Communicating with Data

Fall 2026

Matplotlib

Sarah Groves



Adapted from Hannah Bako and Rafael Alvarado

Reminders

- Due tonight
 - End of unit 1 survey
 - Colormap activity from Wednesday
- Due Sunday: Lab 3

Guiding Principles: steps for building a chart

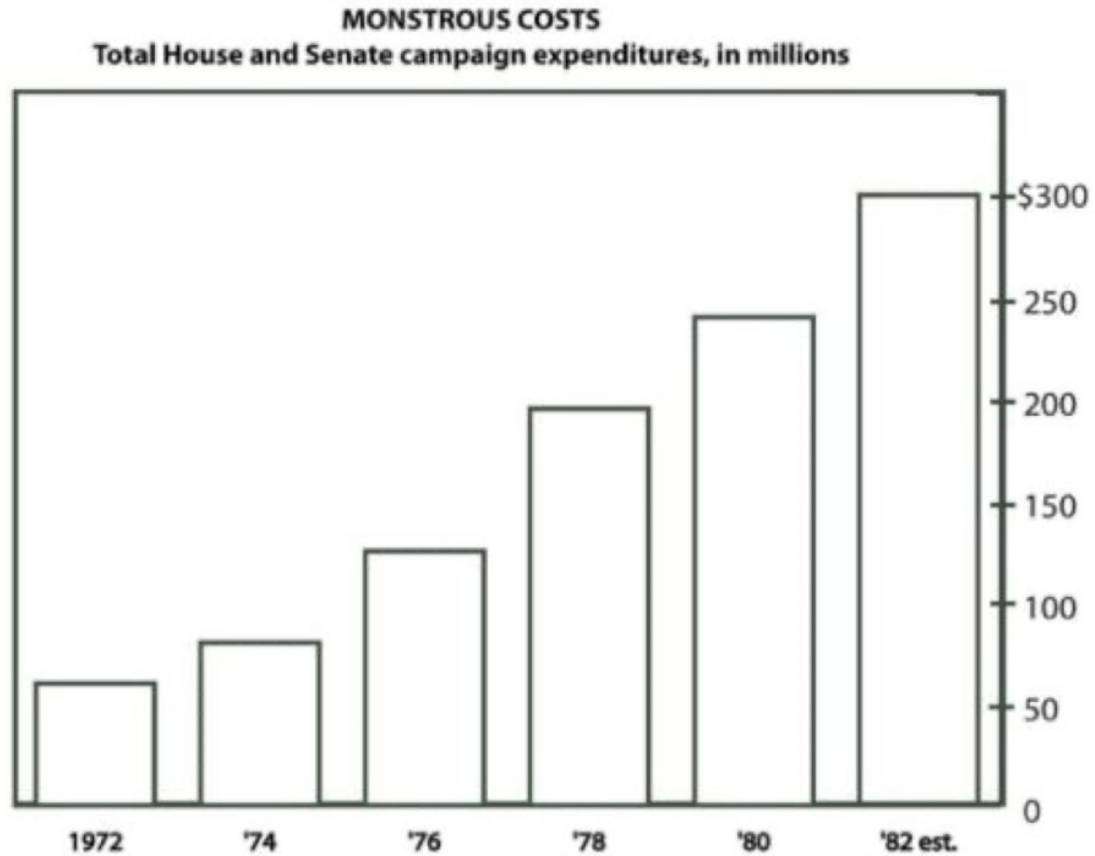
1. First, **identify attribute types** in the data and the **goal** of the graphic.
2. Consider **effectiveness ranking** for attribute type to determine **channel** (identity or magnitude)
 1. Encode most important information with most effective channel first
3. Choose **scales** with appropriate **range** for optimal aspect ratio and **scale type** to change the question being addressed
 1. Consider if you should **start at 0 or something else**
 2. Use well-marked **scale breaks** sparingly
4. Use **hue** for unordered (**nominal**) attributes and sequential or diverging colormaps (hue + **luminance** or **saturation**) for **ordered** attributes
5. Consider the **Lie Factor** of your visualization → should be close to 1

Steps for making an effective and expressive plot

ocument/d/1PC697zQjillf9v
N1B1bVC0gE/edit?usp=sha

**Now that we
know the rules...
we can break the
rules.**

Effectiveness & expressiveness vs. Memorability



Which of the previous
charts was more
effective?

Which was more
memorable?

accuracy in describing the embellished charts was no worse than for plain charts, and their *recall* was significantly better

Matplotlib = "MATLAB Plotting Library"

MATLAB = "Matrix Library"

MATLAB is a once popular programming platform for engineers and scientists to analyze data, develop algorithms, and create models.

Basic plots



`plot([X], Y, [fmt], ...)`
X, Y, fmt, color, marker, linestyle

API



`scatter(X, Y, ...)`
X, Y, [s]izes, [c]olors, marker, cmap

API



`bar[h](x, height, ...)`
x, height, width, bottom, align, color

API



`imshow(Z, ...)`
Z, cmap, interpolation, extent, origin

API



`contour[f]([X], [Y], Z, ...)`
X, Y, Z, levels, colors, extent, origin

API



`pcolormesh([X], [Y], Z, ...)`
X, Y, Z, vmin, vmax, cmap

API



`quiver([X], [Y], U, V, ...)`
X, Y, U, V, C, units, angles

API



`pie(X, ...)`
Z, explode, labels, colors, radius

API



`text(x, y, text, ...)`
x, y, text, va, ha, size, weight, transform

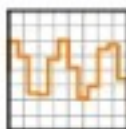
API



`fill[_between](x)(...)`
X, Y1, Y2, color, where

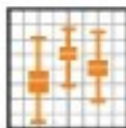
API

Advanced plots



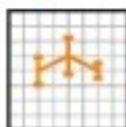
`step(X, Y, [fmt], ...)`
X, Y, fmt, color, marker, where

API



`boxplot(X, ...)`
X, notch, sym, bootstrap, widths

API



`errorbar(X, Y, xerr, yerr, ...)`
X, Y, xerr, yerr, fmt

API



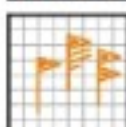
`hist(X, bins, ...)`
X, bins, range, density, weights

API



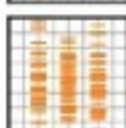
`violinplot(D, ...)`
D, positions, widths, vert

API



`barbs([X], [Y], U, V, ...)`
X, Y, U, V, C, length, pivot, sizes

API



`eventplot(positions, ...)`
positions, orientation, lineoffsets

API



`hexbin(X, Y, C, ...)`
X, Y, C, gridsize, bins

API

Matplotlib is a **powerful** library that can be used to create **many** kinds of plots

These are a **subset** of the plots you can make

Matplotlib can seem **complicated**

Actually, it *is* complicated

But it is also **powerful** and **consistent** in its **design**

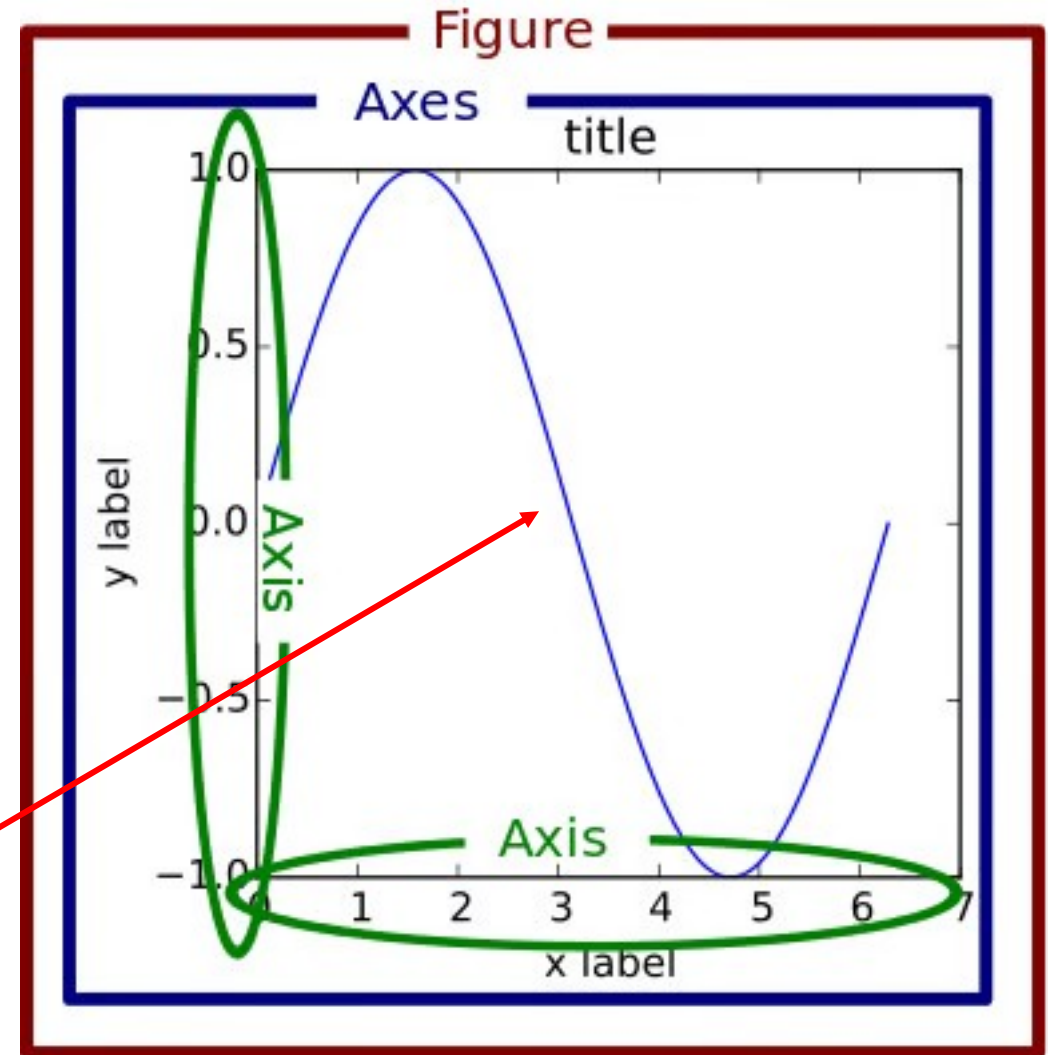
Here are some things to **keep in mind** when learning it

First, everything is based on the ideas
of **Figure** and **Axes**
(and also **plots**)

Basic Structure of a Matplotlib Visualization

- Matplotlib (MPL) has two fundamental concepts:
 - **Figure** and **Axes**
- Here is an example plot showing an axes (a pair of axes) **contained** by a figure
- The **axes** in this case are the x-axis and the y-axis

A **plotted**
line



- **Figures**

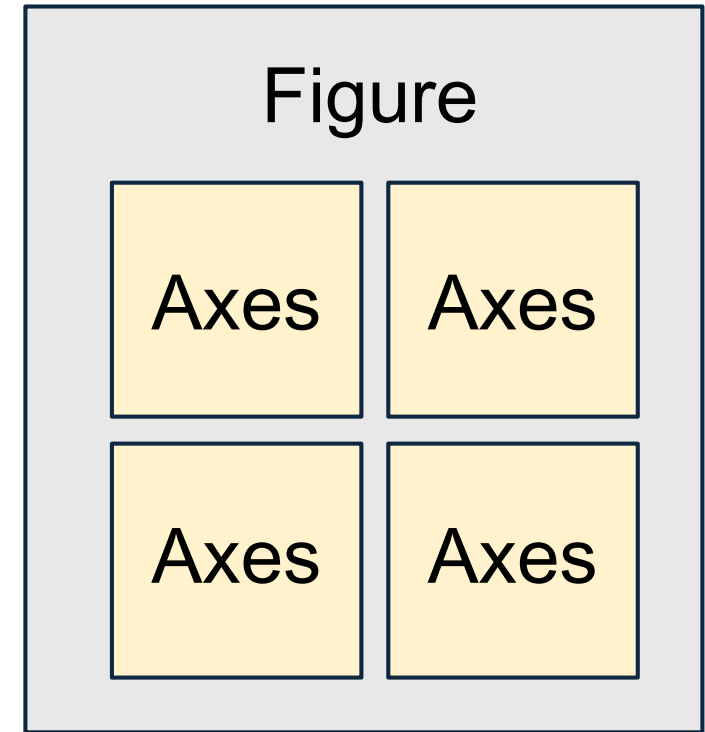
- Top level objects that contains **axes**
Think of this as a canvas

- **Axes**

- Contain all the elements of an individual **plot** in a figure

- **Plot / Plotting**

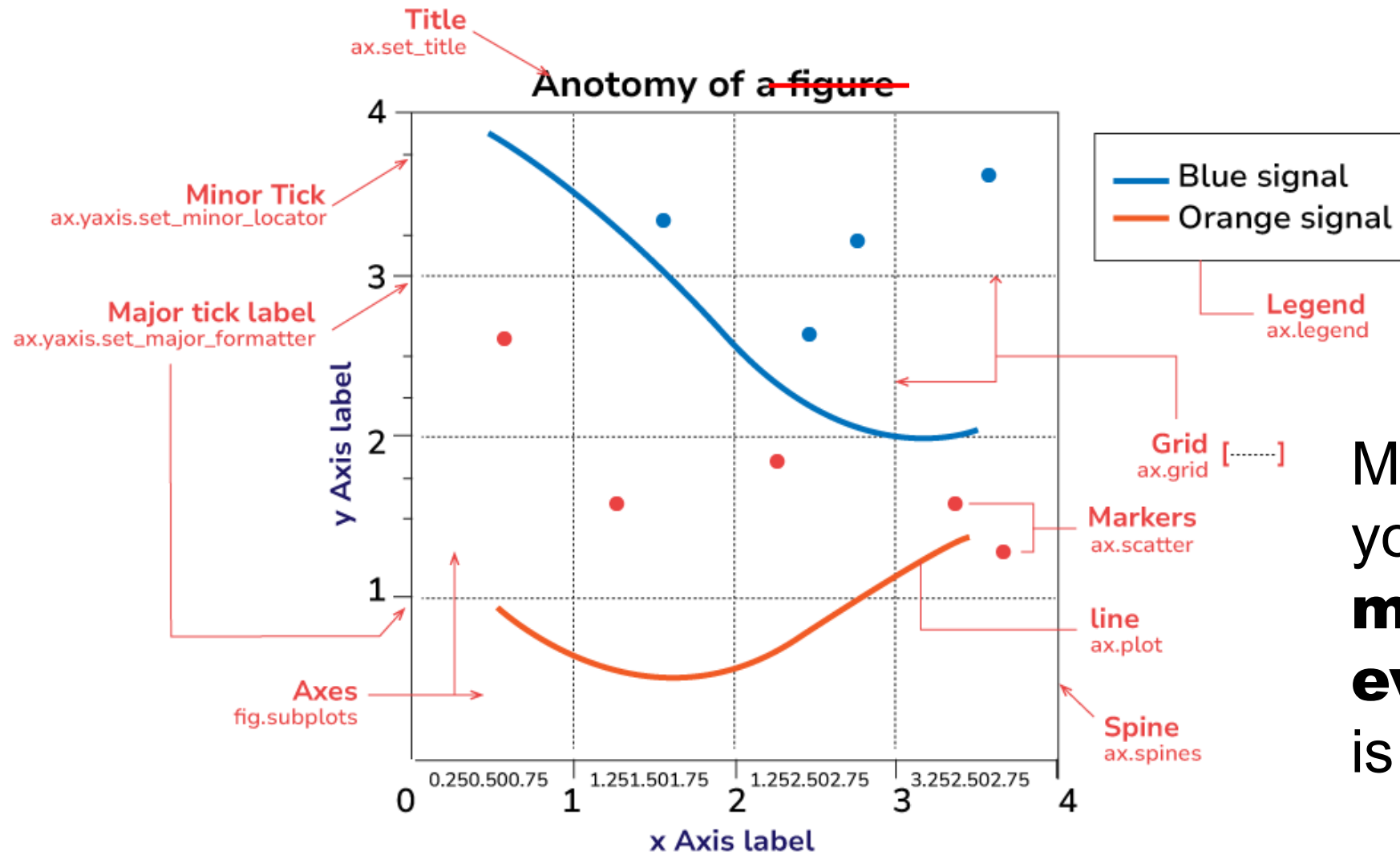
- The actual drawings that go on an axes
E.g. Two lines are two plots
 - Also: the act of plotting



Notebook 1 & 2

Load the data and build the figure and axes

Anatomy of an Axes



Matplotlib enables you to **manipulate everything** that is **named** here

The second thing to keep in mind is the Matplotlib has **two interfaces**

Interactive and **Object Oriented**

In addition, **Pandas** is a wrapper around Matplotlib so it counts as a **third**

This means that example code can be complicated and appear redundant

Sample Code from the Two Interfaces

Operation	Axes interface	pyplot interface
Creating figures	<code>fig, ax = plt.subplots()</code>	<code>plt.subplots()</code>
Plotting data	<code>ax.plot(x, y)</code>	<code>plt.plot(x, y)</code>
Getting properties	<code>label = ax.get_xlabel()</code>	<code>label = plt.xlabel()</code>
Setting properties	<code>ax.set_xlabel("time")</code>	<code>plt.xlabel("time")</code>

Here, **interactive** is called **pyplot**
and **object oriented** is called **Axes**

Anatomy: Steps 1 to 3

Add a scatterplot, title, and labels

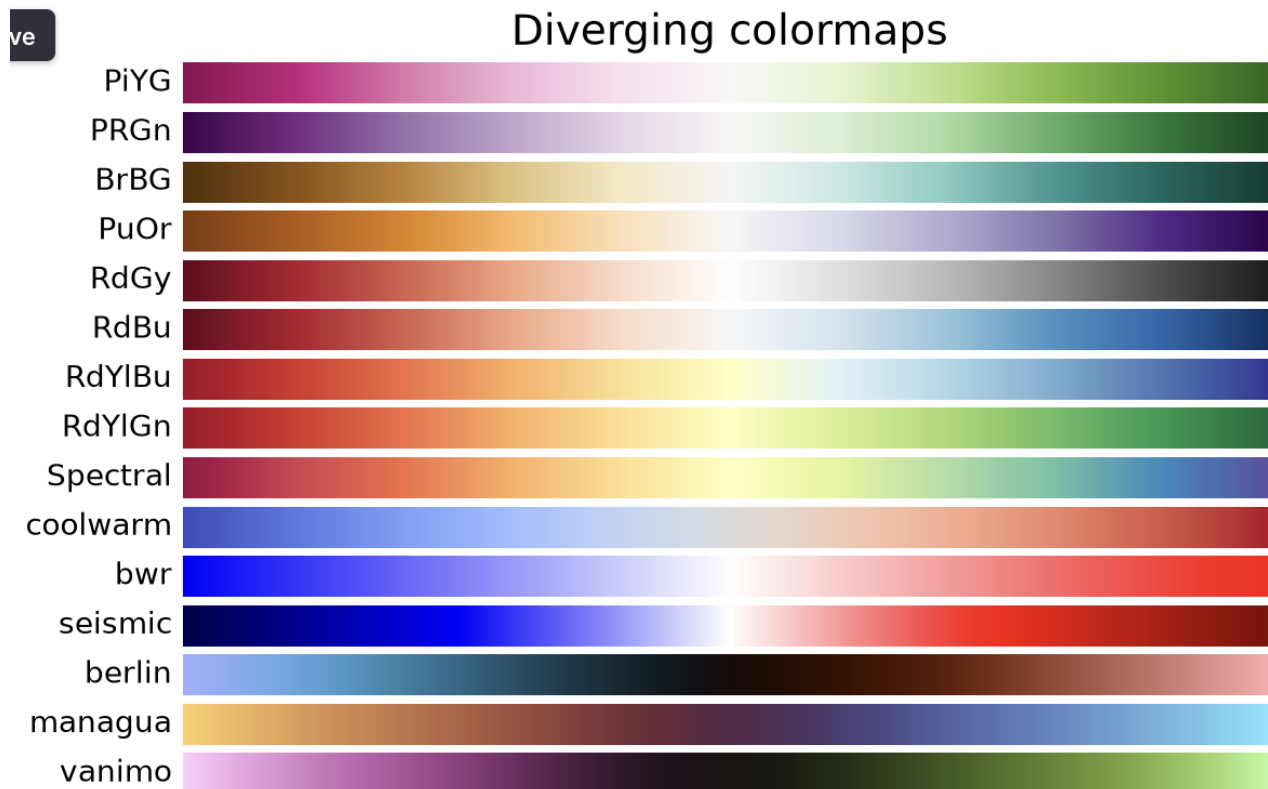
Guidelines for scatterplots and bar plots

Colormaps

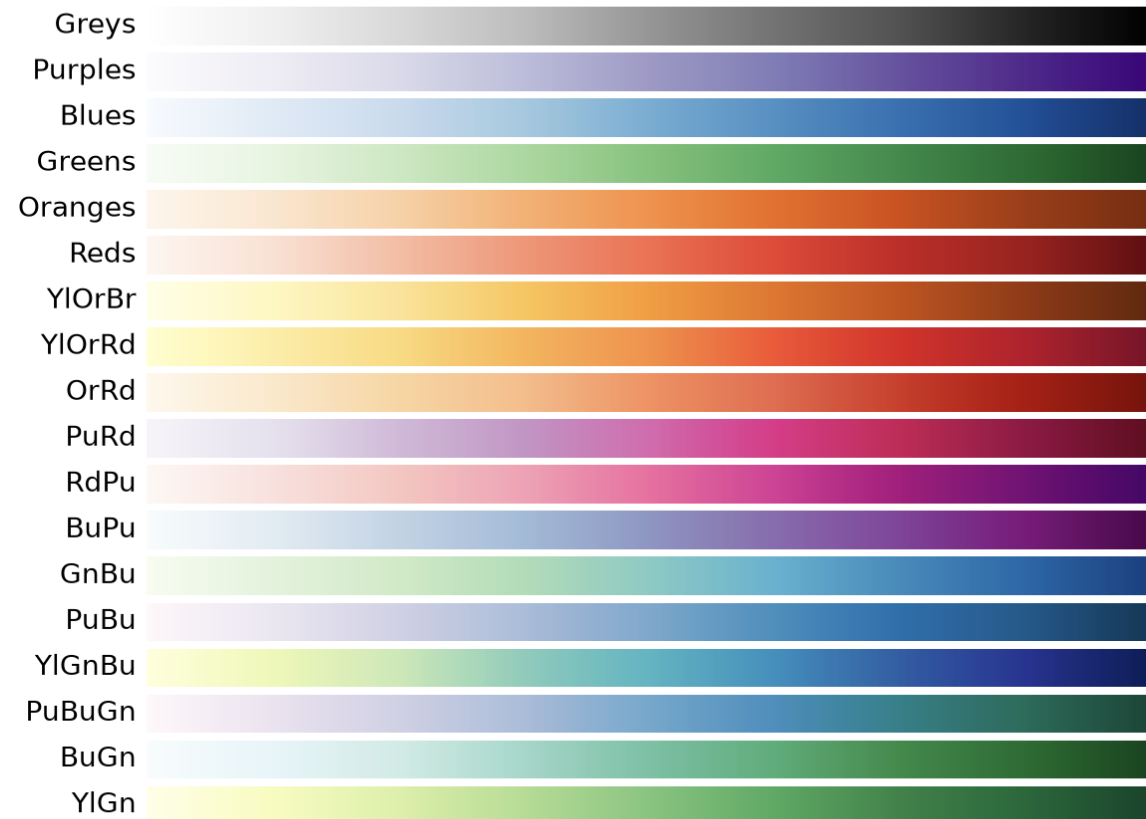
Perceptually Uniform Sequential colormaps



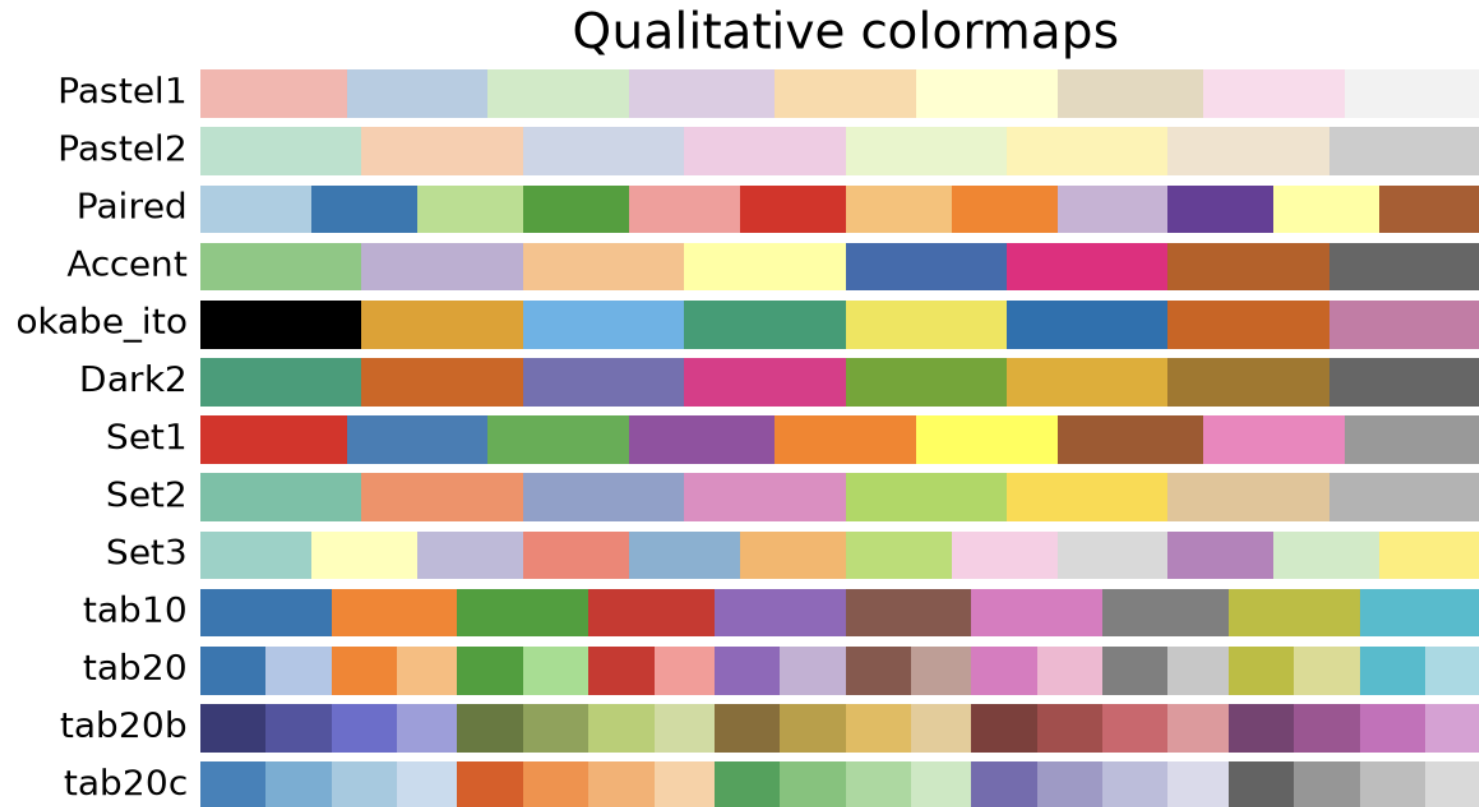
Diverging colormaps



Sequential colormaps

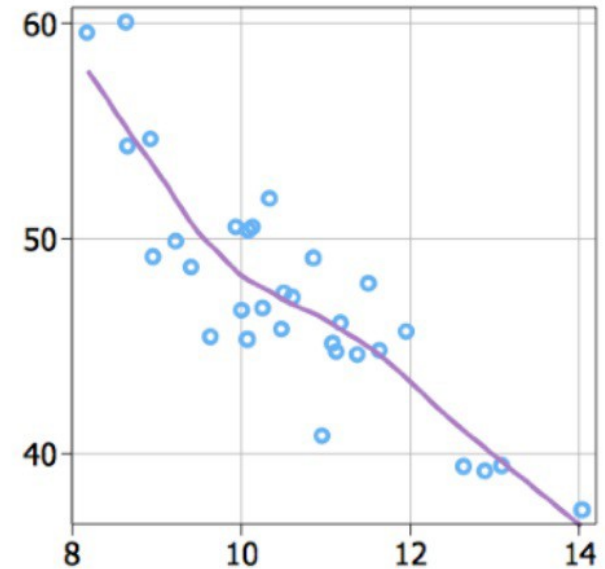
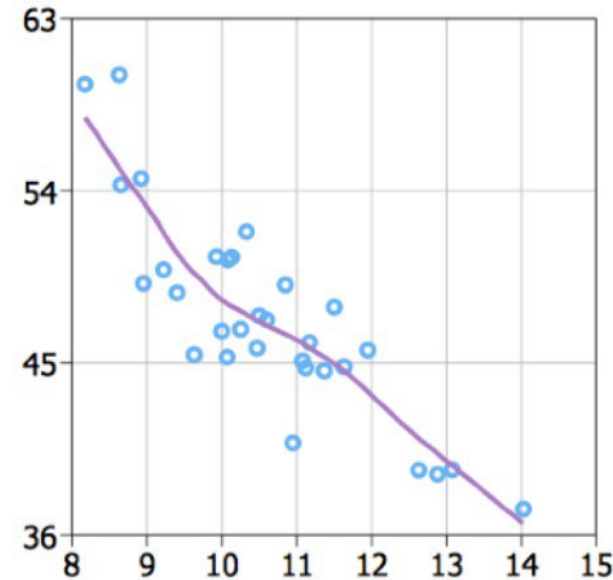
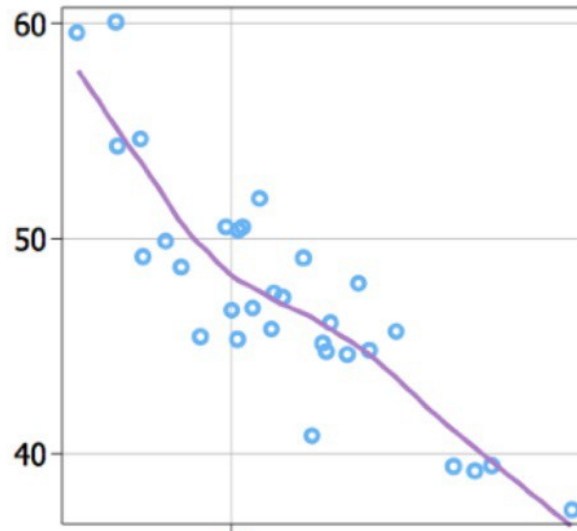
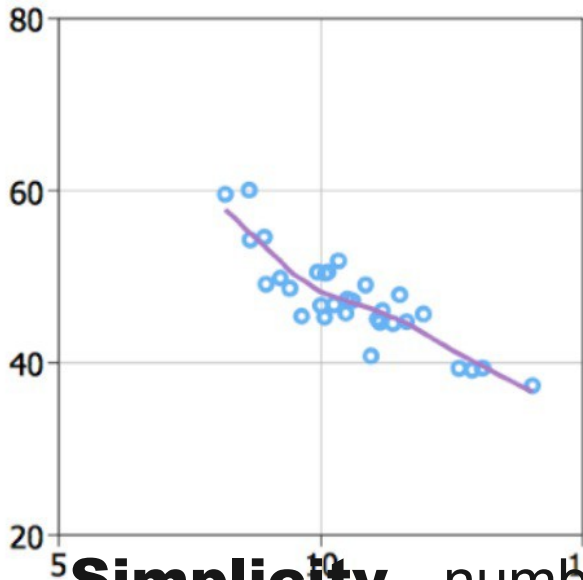


Color palettes for nominal data



Axis Tick Marks

- **Guidelines**



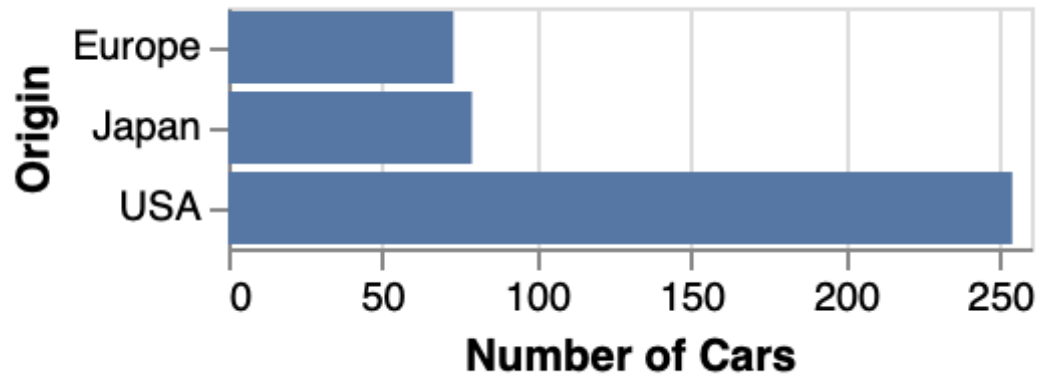
Simplicity - numbers are multiples of 10, 5, 2

Coverage - ticks near the ends of the data

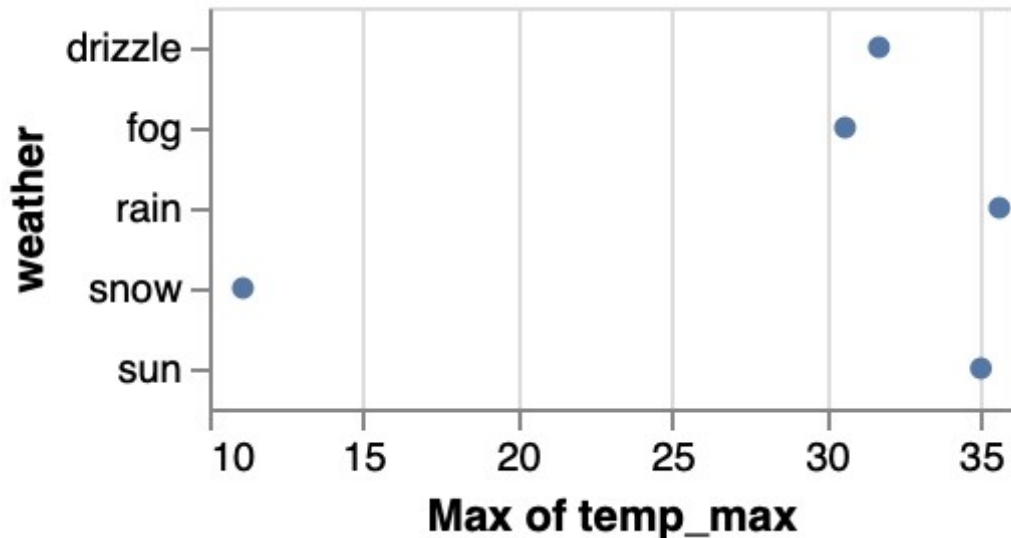
Density - not too many, nor too few

Legibility - whitespace, horizontal text, size

Discrete categories

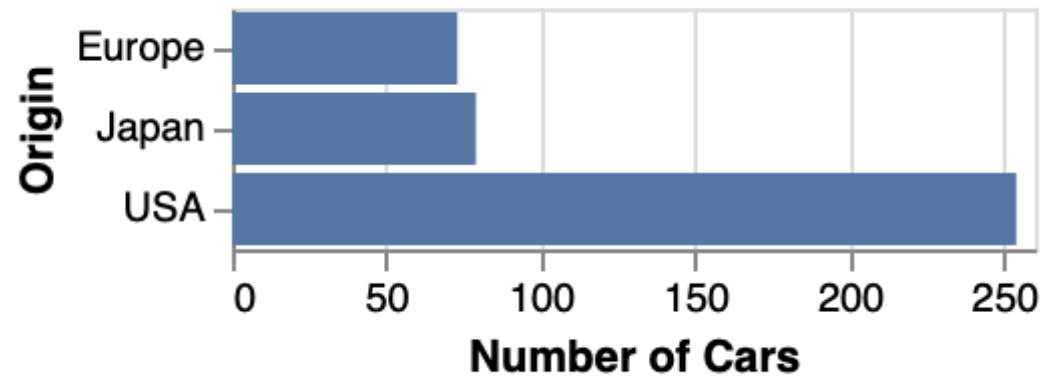
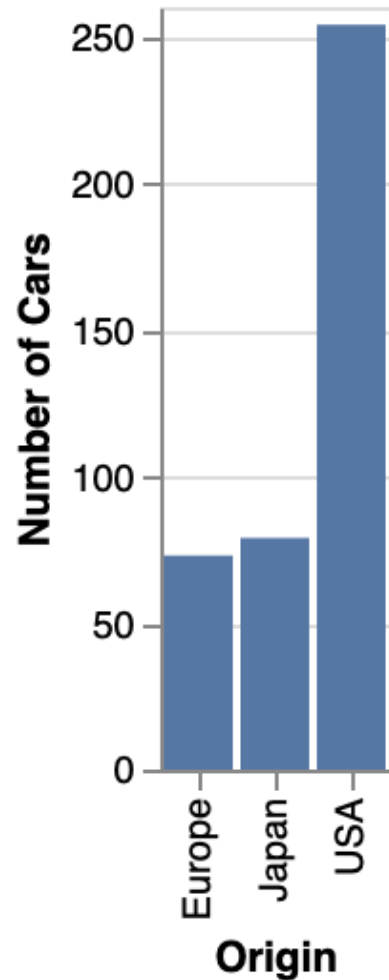


Bars are most effective

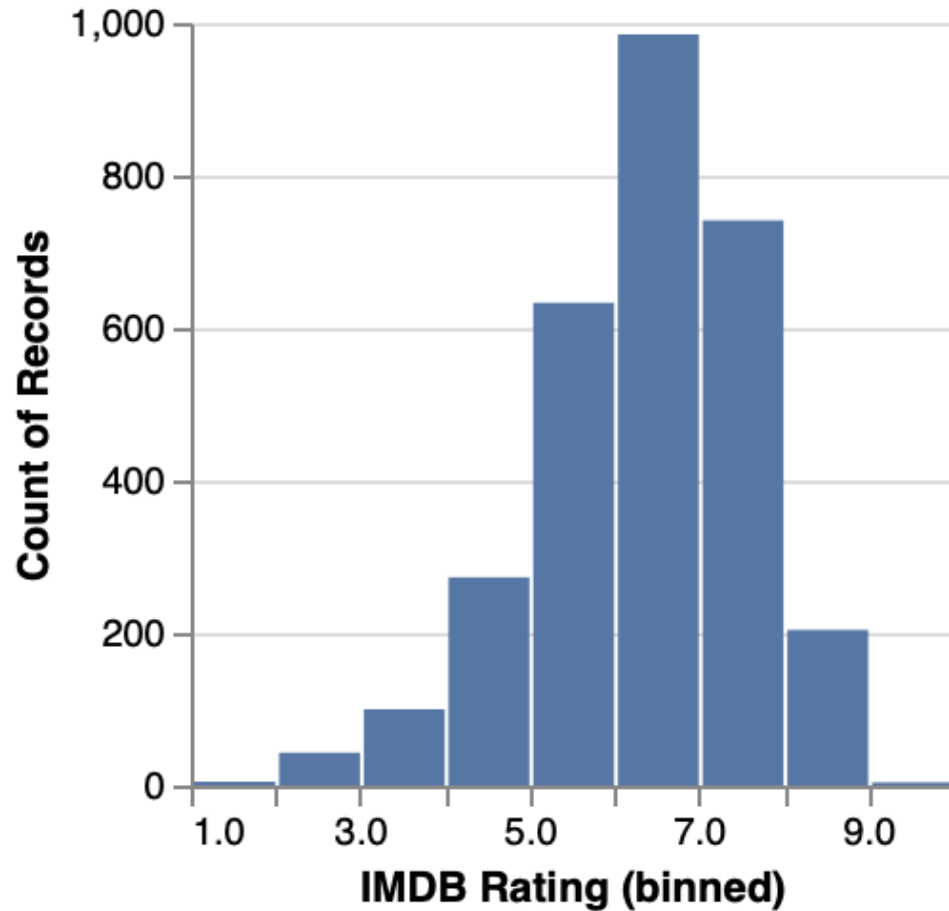


Use points/ticks when measure is not a count

Sideways layout means readable labels



Histograms for distributions



The best way to **learn** it is by **example**
and by **using it** yourself